EXP NO: 9	Text Generation Using Lstm Networks

AIM:

To generate text character by character using an LSTM network trained on the Tiny Shakespeare dataset.

ALGORITHM:

1. Download the Tiny Shakespeare dataset and convert the text to lower case.
2. Create the character dictionaries (character to index and index to character); the text has 39 unique characters.
3. Create input sequences of 40 characters with the next character as the target (50,000 samples).
4. Build the model: Embedding(39, 64), LSTM(128) and a Dense softmax layer with 39 units.
5. Compile with the sparse categorical cross-entropy loss and the Adam optimizer, and train for 1 epoch with batch size 128.
6. Generate text: starting from a seed string, predict the probability distribution of the next character, sample a character from it, append it and repeat for 300 characters.
7. Print the generated text.

CODE:

```
# EXPERIMENT 9
# TEXT GENERATION USING LSTM NETWORKS
# ============================================================

!pip install -q tensorflow numpy

import tensorflow as tf
import numpy as np

print("TensorFlow version:", tf.__version__)

# ============================================================
# STEP 1: LOAD SHAKESPEARE DATASET
# ============================================================

url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.
    txt"

try:
    file_path = tf.keras.utils.get_file(
        "shakespeare.txt",
        url
    )

    with open(file_path, "r", encoding="utf-8") as f:
        text = f.read().lower()

except Exception as e:
    print("Dataset download failed.")
    print("Error:", e)
    raise

print("Dataset loaded successfully.")
print("Total characters:", len(text))

# ============================================================
# STEP 2: CREATE CHARACTER DICTIONARY
```

```
# ============================================================

chars = sorted(set(text))

c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}

print("Number of unique characters:", len(chars))

# ============================================================
# STEP 3: CREATE INPUT AND OUTPUT SEQUENCES
# ============================================================

seq_len = 40

X = []
y = []

# Limit samples so Jupyter runs smoothly
max_samples = min(len(text) - seq_len, 50000)

for i in range(max_samples):

    input_seq = text[i:i + seq_len]
    target_char = text[i + seq_len]

    X.append([c2i[c] for c in input_seq])
    y.append(c2i[target_char])

X = np.array(X, dtype=np.int32)
y = np.array(y, dtype=np.int32)

print("Input shape:", X.shape)
print("Target shape:", y.shape)

# ============================================================
# STEP 4: BUILD LSTM MODEL
# ============================================================

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim=len(chars),
        output_dim=64,
        input_length=seq_len
    ),

    tf.keras.layers.LSTM(128),

    tf.keras.layers.Dense(
        len(chars),
        activation="softmax"
    )
])

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam"
)

model.summary()

# ============================================================
# STEP 5: TRAIN MODEL
```

```
# ============================================================

print("\nTraining started...\n")

history = model.fit(
    X,
    y,
    batch_size=128,
    epochs=1,
    verbose=1
)

print("\nTraining completed successfully!")

# ============================================================
# STEP 6: TEXT GENERATION FUNCTION
# ============================================================

def generate(seed, length=300):

    seed = seed.lower()

    # Keep only characters known by the model
    seed = "".join(c for c in seed if c in c2i)

    if len(seed) == 0:
        seed = "shall i compare thee"

    generated_text = seed

    seq = [c2i[c] for c in seed]

    for _ in range(length):

        current_seq = seq[-seq_len:]

        # Pad if seed is shorter than sequence length
        if len(current_seq) < seq_len:
            current_seq = (
                [0] * (seq_len - len(current_seq))
                + current_seq
            )

        inp = np.array(
            current_seq,
            dtype=np.int32
        ).reshape(1, seq_len)

        prediction = model.predict(
            inp,
            verbose=0
        )[0]

        # Sample according to predicted probability
        next_idx = np.random.choice(
            len(prediction),
            p=prediction
        )

        seq.append(next_idx)

        generated_text += i2c[next_idx]
```

```
    return generated_text

# ============================================================
# STEP 7: GENERATE AND DISPLAY TEXT
# ============================================================

seed_text = "shall i compare thee to a summer's day?"

print("\n" + "=" * 60)
print("GENERATED TEXT")
print("=" * 60)

result = generate(
    seed_text,
    length=300
)

print(result)

print("\n" + "=" * 60)
print("EXPERIMENT 9 COMPLETED")
print("=" * 60)
```

OUTPUT:

```
TensorFlow version: 2.20.0
Downloading data from https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
1115394/1115394 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Dataset loaded successfully.
Total characters: 1115394
Number of unique characters: 39
Input shape: (50000, 40)
Target shape: (50000,)
Model: "sequential"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ embedding (Embedding)           │ ?                      │   0 (unbuilt) │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ lstm (LSTM)                     │ ?                      │   0 (unbuilt) │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense (Dense)                   │ ?                      │   0 (unbuilt) │
└─────────────────────────────────┴────────────────────────┴───────────────┘
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Training started...
391/391 ━━━━━━━━━━━━━━━━━━━━ 49s 118ms/step - loss: 2.6795
Training completed successfully!
============================================================
GENERATED TEXT
============================================================
shall i compare thee to a summer's day?
lourle he onle?
lolt nuts fsot theg alt thmeecgo cil,adi as yhith pthe tiw hanior
kalldns yorser:
e wint bpoutre deccald.
waw
mith wour toum,
the melir medgrid ifiusond poul gasd
theth wous': yf
cictots
an,
wtho og rere. cironnss ,hamtusgs im them
ha wodig erinye ifof.
blaot th scal ong, wors li
============================================================
EXPERIMENT 9 COMPLETED
```

RESULT:

The LSTM character-level text generator was trained for 1 epoch (loss 2.6795) and used to generate 300 characters from the seed text. The output shows character-level structure such as words, spaces and line breaks, but it is not yet coherent because of the very short training.